

CA4

1) Background: the Actel cells

c2 (4:1 mux controlled by {s1,s0})

In `modules.v`, `c2` computes:

- `s1 = A1 | B1`
- `s0 = A0 & B0`

and selects:

- `00 → D00`
- `01 → D01`
- `10 → D10`
- `11 → D11`

So when we want `c2` to behave like a LUT2 with inputs `(A,B)`, we must wire `A1,B1,A0,B0` so that:

- `s1 == A`
- `s0 == B`

c1 (two 2:1 muxes, then final select)

In `modules.v`, `c1` computes:

- `f1 = (SA) ? A1 : A0`
- `f2 = (SB) ? B1 : B0`
- `s2 = S0 | S1`
- `output f = (s2) ? f2 : f1`

So we can use `c1` to implement LUT2 by making:

- `f1(A) = LUT(A, B=0)`
- `f2(A) = LUT(A, B=1)`
- and select between `f1 / f2` using `B` through `S0|S1`.

s1, s2 and FDCP

Both `s1` and `s2` compute a 4:1 mux output `d` (like `c2`) and feed it into `FDCP`:

- `FDCP ff(clk, clr, d, out)`

So both are “mux + flip-flop” cells, with the mux select bits derived from their `A*` / `B*` pins.

2) Extracting LUT inputs from Yosys `$lut`

Yosys emits instances like:

```
$lut #(.LUT(4'hX), .WIDTH(32'h00000002)) inst (  
  .A({sig1, sig0}),  
  .Y(out)  
);
```

or

```
.A(bus[1:0])
```

The mapper’s helper `expand_A_port_for_lut2()` returns **two 1-bit expressions** for `.A(...)`:

- If `.A({X, Y})` → returns `[X, Y]`
- If `.A(bus[1:0])` → returns `[bus[1], bus[0]]`

Then in `map_lut2_to_cell()`:

- `a_signal` = first bit (MSB-like)
- `b_signal` = second bit (LSB-like)

This matches how Yosys constructs a 2-bit vector for `.A`.

3) LUT2 → c2 mapping

Goal

Implement a 2-input LUT with truth table bits `bit0..bit3`:

A	B	Selected bit
0	0	bit0
0	1	bit1
1	0	bit2
1	1	bit3

Wiring strategy

We force `c2`'s select lines to equal `{A,B}` :

- `s1 = A1 | B1 = A`
- `s0 = A0 & B0 = B`

We achieve that using:

- `A1 = 1'b0`
- `B1 = A`
- `A0 = B`
- `B0 = 1'b1`

Code (from mapper)

```
# LUT2 -> c2
return (
    f"c2 {lut['name']} (\n"
    f"    .D00(1'b{bit0}), .D01(1'b{bit1}), .D10(1'b{bit2}),
    .D11(1'b{bit3}),\n"
    f"    .A1(1'b0), .B1({a_signal}), .A0({b_signal}), .B0(1'b1),\n"
    f"    .out({lut['output']})\n"
    f");"
)
```

4) LUT2 → c1 mapping

Strategy

`c1` can be used as a 2:1 mux between two sub-functions `f1` and `f2` :

- `f1` is selected when `S0 | S1 = 0`

- f_2 is selected when $S_0|S_1 = 1$

So we build:

- $f_1(A) = \text{LUT}(A, B=0)$
- $f_2(A) = \text{LUT}(A, B=1)$

Then choose between them using B by setting $S_0=B$ and $S_1=B$.

Code (from mapper)

```
# LUT2 -> c1
return (
    f"c1 {lut['name']} (\n"
    f"    .A0(1'b{bit0}), .A1(1'b{bit2}), .SA({a_signal}),\n"
    f"    .B0(1'b{bit1}), .B1(1'b{bit3}), .SB({a_signal}),\n"
    f"    .S0({b_signal}), .S1({b_signal}), .f({lut['output']})\n"
    f");"
)
```

5) LUT1 → c2 mapping

A 1-input LUT has:

- bit0 for $A=0$
- bit1 for $A=1$

We want the output to depend only on a .

Strategy

Make c_2 select depend only on a such that:

- $a=0 \rightarrow \{s_1, s_0\}=00 \rightarrow \text{select } D_{00}$
- $a=1 \rightarrow \{s_1, s_0\}=11 \rightarrow \text{select } D_{11}$

We reuse the same select wiring idea, but drive both select bits from a :

- $s_1 = A_1|B_1 = a$ via $A_1=0, B_1=a$
- $s_0 = A_0\&B_0 = a$ via $A_0=a, B_0=1$

Code (from mapper)

```
# LUT1 -> c2
return (
    f"c2 {lut['name']} (\n"
    f"    .D00(1'b{bit0}), .D01(1'b{bit0}), .D10(1'b{bit0}),
    .D11(1'b{bit1}),\n"
    f"    .A1(1'b0), .B1({a}), .A0({a}), .B0(1'b1),\n"
    f"    .out({lut['output']})\n"
    f");"
)
```

(Only D00 and D11 matter with this select wiring; the other entries are set safely.)

6) LUT1 → c1 mapping

Strategy

Use only the A-side mux (f1), and force selection to always pick f1:

- $f1 = (SA ? A1 : A0)$ with $SA=a$, $A0=bit0$, $A1=bit1$
- force $S0|S1=0$ by setting $S0=0$, $S1=0$

Code (from mapper)

```
# LUT1 -> c1
return (
    f"c1 {lut['name']} (\n"
    f"    .A0(1'b{bit0}), .A1(1'b{bit1}), .SA({a}),\n"
    f"    .B0(1'b0), .B1(1'b0), .SB(1'b0),\n"
    f"    .S0(1'b0), .S1(1'b0), .f({lut['output']})\n"
    f");"
)
```

7) DFF (\$_DFF_P_) → s1 mapping

$\$_DFF_P_$ is a positive-edge D flip-flop with no enable and no reset.

Strategy

Force the internal mux to always pick D00, and set $D00=D$.

- In `s1` the select bits are:
 - `s1 = A1 | B1`
 - `s0 = A0 & clr`
- If we set `A1=B1=0` then `s1=0`
- If we also set `A0=0` and `clr=0`, then `s0=0`
- So `{s1,s0}=00` and the mux output becomes `D00`.

Code (from mapper)

```
return (
  f"s1 {inst} (\n"
  f"    .D00({D}), .D01({D}), .D10({D}), .D11({D}),\n"
  f"    .A1(1'b0), .B1(1'b0), .A0(1'b0),\n"
  f"    .clr(1'b0), .clk({C}), .out({Q})\n"
  f");"
)
```

8) SDFFE (`$_SDFFE_PP0P_`) → `s2` mapping (CE + async reset-to-0)

This cell has:

- `C` : posedge clock
- `E` : clock enable (active high)
- `R` : reset (active high)
- reset value = 0

IMPORTANT NOTE (why we do NOT use `clk & E`)

Clock gating (`clk & E`) is unsafe if `E` changes while `clk=1` (it can create a spurious posedge). Your testbench pulses `start` for a few ns while `clk` is running, so this hazard can occur.

So we implement enable as a **data mux**:

- when `E=0` → hold: `D_ff = Q`
- when `E=1` → load: `D_ff = D`

Strategy

Use `s2` as a 2:1 mux controlled by `E` :

- Force `s1=0` by setting `A1=B1=0`
- Make `s0=E` by setting `A0=E, B0=1`

Then:

- `{s1,s0}=00` when `E=0` → select `D00=Q` (hold)
- `{s1,s0}=01` when `E=1` → select `D01=D` (load)

Reset is handled by `FDCP` via `clr=R`.

Code (from mapper)

```
return (
    f"s2 {inst_name} (\n"
    f"    .D00({Q}), .D01({D}), .D10({D}), .D11({D}),\n"
    f"    .A1(1'b0), .B1(1'b0), .A0({E}), .B0(1'b1),\n"
    f"    .clr({R}), .clk({C}), .out({Q})\n"
    f");"
)
```

10) Number of cells according to the mapper output:

- Found 2 LUT1 and 746 LUT2 instances (out of 748 \$lut)
- Total number of c2: 748
- Total number of S1: 0
- Total number of S2: 92

11) Pipeline script :

- automates the flow of running yosys and then giving the output to the python lut2mapper.py so you get the mapped_top and modules just bby running a single script.